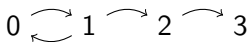


All for one and none forall: Compiling polymorphic relations without monomorphization

Dmitri Volkov, Yafei Yang, Chung-chieh Shan

2026 August 24



```

(defrel (graph
  (from : (Num 4)) (to : (Num 4)))
  (disj
    (conj (== from 0) (== to 1))
    (conj (== from 1) (== to 2))
    (conj (== from 1) (== to 0))
    (conj (== from 2) (== to 3))))

```

		to			
		0	1	2	3
from	0	0	1	0	0
	1	1	0	1	0
	2	0	0	0	1
	3	0	0	0	0

Represent graphs as relations or as adjacency matrices.

semiringKanren computes matrices for arbitrary relations!

Built-in/primitive relations

(== x y)

		y			
		0	1	2	3
x	0	1	0	0	0
	1	0	1	0	0
	2	0	0	1	0
	3	0	0	0	1

(=/= x y)

		y			
		0	1	2	3
x	0	0	1	1	1
	1	1	0	1	1
	2	1	1	0	1
	3	1	1	1	0

disj is addition

```
(defrel (r0 (x : (Num 4)) (y : (Num 4)))  
  (disj  
    (== x 0)  
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

conj is multiplication

```
(defrel (r1 (x : (Num 4)) (y : (Num 4)))  
  (disj  
    (== x 0)  
    (== y 0)))
```

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

fresh is summation

```
(defrel (r2 (y : (Num 4)))  
  (fresh (x : (Num 4))  
    (conj (== x 0) (=/= y x))))
```

$$\sum_x \left(\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} * \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix} \right) = \sum_x \begin{bmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \\ = \begin{bmatrix} 0 & 1 & 1 & 1 \end{bmatrix}$$

semiringKanren values inhabit algebraic data types.

```
sole : Unit [1]
(left sole) : (Sum Unit (Num 4)) [1 0 0 0 0]
(right 0) : (Sum Unit (Num 4)) [0 1 0 0 0]
      ⋮
(right 3) : (Sum Unit (Num 4)) [0 0 0 0 1]
```

(Num n) is shorthand for a sum type of n Units
(semiringKanren also supports product types)

$$\begin{bmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

